

ArchSync: Technical Evidence Supplement

ARCHSYNC AUTHOR TEAM

A REPRODUCTION AND INTERPRETATION DETAILS

This appendix supplies operational detail for the single-column review profile. It is supplied as a separate supplement with the compact profile; acceptance of supplementary uploads is not established by the public instructions. Its purpose is to make the development evidence inspectable, not to add independent observations.

A.1 Keep historical and current evidence separate

Historical D1/D2/P3 observations are reproduced from their pinned benchmark inputs and packaged analyzers. The reporting derivation uses benchmark commit 24d63ebf2fc3075a1d64f1eaff38cdc0b7f586fb. Its ground truth, Phase 2 and Phase 3 records have individual hashes. Rerunning a current analyzer on that same corpus answers a different question: whether the updated implementation preserves known development behavior. The latter cannot silently replace the former in a before/after table.

The artifact therefore distinguishes original packages, current source snapshots, current distribution builds, first failing runs, repaired runs and unchanged historical result inputs. Current source and package hashes identify the evaluated candidate; an unreleased version string alone does not identify its bytes. The accompanying manifest maps each new manuscript count to its raw receipt. A successful package installation is an operational compatibility check, not an independent assessment of architectural correctness.

The historical evidence-location decomposition is explicitly post-hoc. Nine finding-bearing cases concern call sites; cases 08 and 17 concern deterministic anchors for missing edge/path requirements. Case 15 contains two violated rules but counts once. A match in these finite cases cannot establish that all useful evidence was found, correctly ranked or understood by a developer.

A.2 Finite oracle construction

For three nodes, six ordered non-self node pairs and two edge types give 12 binary edge slots, hence $2^{12} = 4096$ graphs. For each graph, an adjacency matrix is built independently for all edges and for each individual edge type. Direct requirements query that matrix; path requirements query its transitive closure. The diagonal is not initially true: a required path must contain an edge. A cycle may nevertheless establish a nonempty path back to its starting node.

For each of four rule kinds, the source and target independently range over three exact identifiers and a full wildcard, and the edge-type restriction ranges over unrestricted and the two types. Thus $4 \times 4 \times 4 \times 3 = 192$ rule configurations are checked for every graph. The oracle compares per-rule violation counts, aggregate violations and classification. It imports no production graph-building, selector-matching or traversal helper. This structural separation reduces direct implementation reuse but does not establish independent authorship or correctness of the shared rule interpretation.

Ordering and consistent-renaming checks use masks 0 through 4080 in steps of 17, giving 241 graph states. Inverse-delta checks compare every graph with the empty-edge graph. They verify that added edges in one direction are removed edges in the other, and that edge changes without rules classify as evolution. These checks do not enumerate

arbitrary pairs of graphs, metadata changes, arbitrary wildcard fragments or source-language programs. The chosen domain is exhaustive only within its declared bounds.

A.3 Mutation procedure and retained failures

The mutation harness copies the relevant source and test into a temporary isolated workspace, changes one declared expression at a time, and invokes the same test suite with a fixed timeout. Production source remains unchanged by this procedure. Every run retains the replacement text, original and mutated source hashes, test hash, command output, exit state and native report. A passing unmodified baseline is required before mutant outcomes are interpreted.

Mutant states distinguish test passes, assertion failures and execution errors. The eight faults disable deny enforcement, invert allow logic, invert direct-require satisfaction, ignore path edge types, prevent traversal beyond one edge, miss path target detection, demote violation, or demote evolution. The scope is intentionally narrow. A compile error or timeout is not silently counted as an assertion kill, and no claim about all equivalent mutants is made.

The original surviving classification mutation matters: it demonstrated that correct finding counts alone did not guarantee the test checked the returned decision. After the assertion was added, the same fault was detected. Reporting only the final perfect score would hide that development intervention. Both versions of the test evidence remain available, so reviewers can distinguish test strengthening from confirmation on an untouched sample.

A.4 Detector repairs and residual uncertainty

The first adversarial set contains 18 cases: 15 failed before intervention and all 18 passed after it. Positive controls check aliases, captured resources and an unshadowed global fetch alongside local bindings. Type-only import tests deliberately include source that a TypeScript compilation gate would reject. They test the analyzer’s robustness when parsing source without that gate, not the prevalence of such constructs in real projects.

A later intervention added eight cases for concatenation and reassignment: seven failed in the cumulative 26-case pre-repair suite. Three further boundary controls were added after repair, yielding 29 passing adversarial cases. These before/after suites therefore have different total sizes, and a naive percentage comparison would be misleading. An old positive fixture concatenated a suffix into an invalid URL port; the fixture was corrected to a valid path suffix and the invalid-port input retained as a negative regression.

A third intervention adds 14 cases to the 29-case suite: 12 write scenarios and two controls for unrelated or shadowed bindings. The identical 43-case suite has 11 failures and 32 passes before the change, then 43 passes afterward. One added write scenario already passed. The repair resolves variable identities within array/object destructuring (including rest/default targets), increment/decrement operations and assignment targets in for-in/for-of loops. For example, a URL binding overwritten by an array assignment must not retain its initializer as the target of a later fetch. These constructed programs test specified behavior; they are not independent observations of real-world prevalence.

Recognized-write invalidation is conservative and file-wide. It can suppress a genuine call that appears before re-assignment; it does not implement statement-order data flow. Property mutation, imported constructor rebinding, dynamically built templates and alternative fallback destinations remain separate risks. When no edge is emitted, the tool does not prove that no runtime relationship exists. Confidence constants attached to emitted evidence cannot resolve this missing-observation uncertainty. Analyzer version 0.4 invalidates caches from earlier semantics. The full repository gate passed from a clean temporary candidate whose 410 tracked files matched the working source byte-for-byte. The first full run stopped at a CLI assertion still expecting version 0.3; that failure is retained alongside the corrected assertion and passing rerun. These are packaging and regression checks, not independent reproduction.

A.5 Incremental and cache experiment boundary

The incremental suite creates temporary Git repositories and checks additions, modifications, deletions, renames within and between components, and complete-component deletion. Cold and warm results are compared with fresh full scans. The initial nine adversarial cases all failed on the original implementation; a later ignored-source case exposed a further false PASS. Before and after receipts preserve those observations.

Most differential cases compare classification, decisions, finding count and scanned-file count. Cold/warm comparisons additionally inspect architecture deltas and introduced/resolved findings. The nested-component case compares the complete observed graph and evidence. This distinction is important: aggregate count agreement alone does not prove identity of every graph edge. The suite does not constitute a proof over all Git configurations or filesystem layouts.

The baseline cache now separates project-relative roots, architecture identity and analyzer semantics. A payload hash rejects accidentally changed JSON; concurrent writers use distinct temporary publication paths. Neither mechanism authenticates a malicious cache writer. Git inventory and scanning are not an atomic snapshot, so the worktree must remain stable during a check. Ignored source outside excluded build/dependency directories is included because full scanning would read it; the gate must not ignore a relationship merely because Git does.

A.6 Independent follow-up design

The technical extension specifies a future repository frame selected before prediction inspection, with lineage-aware development/pilot/holdout separation. Two independent human reviewers must enumerate supported relations across the complete declared source scope, seal their separate labels, and preserve disagreements before adjudication. Checking emitted predictions alone can estimate precision but cannot establish recall. Supported-vocabulary coverage, execution failures and unresolved labels require separate accounting.

A comparator must share a nonempty, frozen observation unit with ArchSync. An import dependency and a remote service interaction are not interchangeable merely because both can be drawn as graph edges. Performance experiments must pair the same workloads and machines, control the graph-cache condition, separate priming from timed execution and retain failures. Operating-system disk cache is not equivalent to the ArchSync graph cache. These protocols are proposed technical designs, not completed studies or evidence of independent human participation.